

CHAPTER 1: INTRODUCTION

This chapter talks about the ideas of Cyber-Physical Systems and gives us the background we need to understand the project. It tells us how Cyber-Physical Systems combine computers, communication networks and physical things to make systems that can watch analyse and react to what's happening in the real world right now. We learn about how sensors, controllers and actuators work to make a closed-loop system that can control things accurately and on time.

The chapter also looks at how Cyber-Physical Systems have changed over time from control systems that worked alone to modern systems that are connected to the internet and can talk to each other using wireless technology. We see how things like the Internet of Things, cloud computing and artificial intelligence have helped make Cyber-Physical Systems smarter and better, at adapting to things.

Then we learn about what the project's trying to do which is to design and build a robotic arm that can be controlled using special hardware and feedback that happens in real time. We find out how sensors, servo motors and a microcontroller all work together and how ROS 2 is used to help them talk to each other and simulate things. We get to know the parts and tools used in the project both in the hardware and the software so we can understand how the whole system is put together and made.

1.1 An overview of Cyber-Physical Systems

Advanced Cyber-Physical Systems, also known as CPS are incredibly advanced systems that combine computational algorithms with physical objects or processes, all of which are connected by communication networks (e.g. the internet). In CPS, embedded computers and software run programs that monitor and control physical processes through feedback loops where physical processes can influence how calculations are performed and vice-versa [1]. CPS combine sensing, computing, controlling and networking into a single system, providing for real-time interaction between the digital domain (cyber) and the physical domain (real world).

The purpose of implementing CPS is to improve efficiency, reliability and automation of different types of applications.

In a CPS application, data from the physical environment is captured via sensors and that data will then be processed through embedded computing units and then based upon the data processed by the embedded computing units, control commands will be developed and then executed through actuators to affect the physical system to which they are connected to. [1]

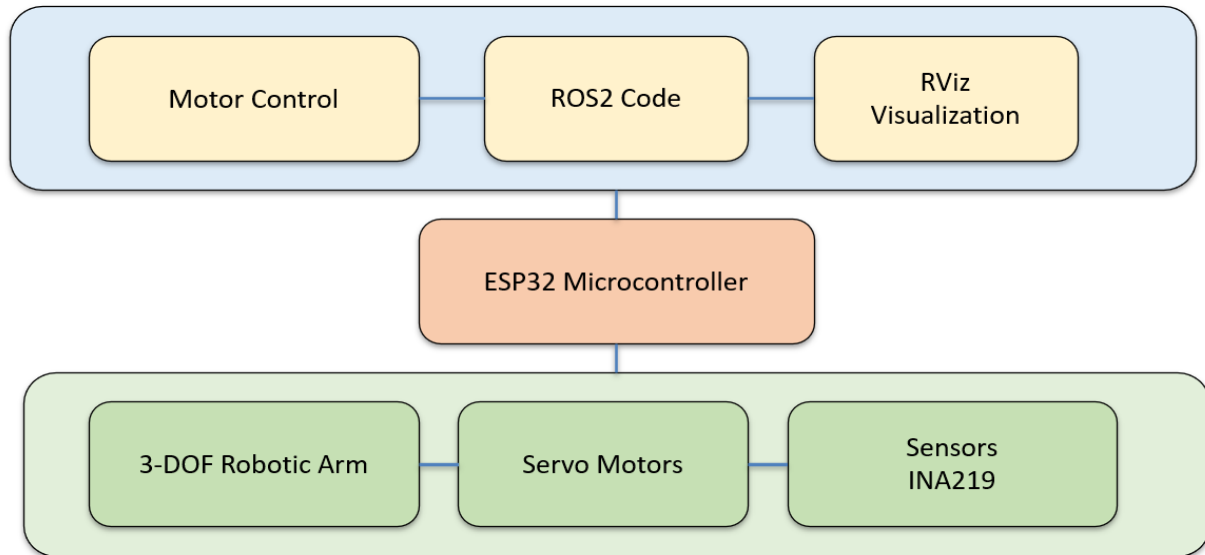


Figure 1.1: Block diagram of map of cyber physical system

1.2 Layers of Cyber Physical System

A Cyber-Physical System (CPS) is organized into three interconnected layers that together enable seamless interaction between the physical world and digital intelligence, forming a continuous feedback loop for real-time monitoring and control.

The Physical Layer is the foundation of a CPS, comprising all tangible components that interact directly with the real world. It includes sensors that measure physical quantities such as temperature, pressure, and motion, as well as actuators like motors, valves and robotic arms that carry out physical actions based on received commands. Embedded systems and PLCs are integrated into machinery to execute low-level control logic. Since this layer operates in direct contact with the physical environment, it demands strict real-time performance and any failure here has immediate real-world consequences.

The Network and Communication Layer acts as the bridge between the physical and cyber domains, ensuring that data is transmitted reliably, securely, and with minimal latency. It encompasses both wireless protocols such as Wi-Fi, Zigbee and 5G and industrial wired

standards like Modbus and OPC-UA. Edge computing devices pre-process sensor data locally to reduce bandwidth usage and improve response times, while security mechanisms such as encryption and authentication protect data in transit. The key challenges at this layer are maintaining low latency, high reliability, and scalability across large deployments.

The Cyber Layer is the intelligence of the system, where collected data is processed, analysed and converted into actionable decisions. It includes cloud platforms for large-scale data storage and processing, digital twins for real-time simulation and predictive maintenance, and AI and machine learning models for anomaly detection and performance optimization. Control algorithms continuously compute commands and dispatch them back to the physical layer, while dashboards and Human-Machine Interfaces allow operators to monitor the system and intervene when needed.

The three layers work in a continuous bidirectional cycle data flows upward from the physical environment through the network to the cyber layer for analysis, and decisions flow back down to actuators in the physical world. This closed-loop interaction is what distinguishes a CPS from a conventional embedded system, enabling it to reason intelligently and act precisely in real time.

1.3 Evolution of CPS

The 1980s to the 1990s saw the development of embedded systems. The physical devices integrated with microcontrollers (the device used to perform control of process) allowed for greater automation and efficiency in operations. However, during this time period, the communication between embedded systems and other systems was minimal.

The late 1990s to early 2000s brought the establishment of networked embedded systems as the result of technological advances in the areas of the internet and wireless technologies which enabled the ability to communicate between physical devices and allow for real-time monitoring and control.

The introduction of the Internet of Things (IoT), cloud computing and data analytics during the 2010s provided considerable change to CPS. The systems became highly interconnected (meaning that data was able to be shared among multiple physical devices) and capable of making decisions in real time. More recently, developments in the areas of artificial intelligence and machine learning will make CPS systems more intelligent, adaptive and autonomous.

1.4 Scope of Project

This project develops a real-time Cyber-Physical System (CPS) to control a robotic arm by combining hardware components and embedded computing. It connects sensors, motors, and a microcontroller so that the movement of the robotic arm is controlled through programmed instructions.

Sensors collect data such as position and force, and this data is processed by the controller to decide how the arm should move. Based on this, motors are activated to perform actions like picking and placing objects.

The system ensures that mechanical movement and computation work together at the same time, instead of separately. As soon as data is received, the system responds immediately, allowing smooth and precise operation.

A single robotic limb takes shape, built with five moving parts guided by small SG90 servos for accurate positioning. From the middle of it all, an ESP32 chip runs operations - reading data, then sending out movement orders to each joint. Instead of guessing effort levels, the machine senses resistance through either a load sensor or electric draw, measuring how hard muscles work when grabbing things.

Much of the work focuses on bringing together ROS 2, linking hardware and software so they talk clearly. Because of this setup, live data moves smoothly, joints report positions, systems stay watched. On another path, a copy of the robot arm takes shape through URDF plus Xacro (XML Macros) file, built not from metal but code.

This ROS 2 version lives inside simulations, shows motion, runs checks long before any machine powers up.

Starting off differently each time, the work looks into ways of handling data - using tools like the Kalman filter to clean up shaky torque readings so results become more trustworthy. But this shift in approach, choices tied to sorting by weight, tweaking how tightly something is held or checking if the machine runs well rest on firmer ground.

Major scope includes:

- Development of Arduino and Python based control algorithms.
- Real-time feedback system for improved precision.
- Integration of hardware components like sensors, motors and communication modules.

- Use of 3D printing technology for designing and fabricating robotic arm components.

When it comes to real-world use, this setup should branch into specific areas:

- Industrial automation and smart manufacturing.
- Warehouse management and sorting systems. [2]
- Assistive robotics and human-robot collaboration. [3]

1.5 Components of the Project

The project utilizes the ESP32 microcontroller to take on the role of the central controller within the system, taking responsibility for processing and interfacing the hardware and software of the project. The built in Wi-Fi and Bluetooth allows the system to communicate externally, and enables the project to be interfaced to more advanced systems such as ROS2 and to gain extra functionality other than basic control enabling support for more advanced robotics applications.[4] An SG90 servo motor was utilized to control the movement; this has the necessary precision of angular control required to control the robotic gripper by accurately actuating to grip or release objects via the two jaws. Connection between the ESP32 and the servo motor was done via a breadboard and jumper wires; this system allowed for quick prototyping of circuits without the need for any soldering, making it easy to test and modify the circuit while debugging.

The mechanical part of the project consists of robotic links and parts of the robotic gripper, designed and 3D printed. These were utilized as the physical structure for the robotic arm and gripper to interact with real world elements.

The robotic parts were designed in Fusion 360 to provide accurate custom parts that were correct for fitting and interaction upon printing.[5] In terms of software the project was developed in Ubuntu OS, which gives a stable compatible system for applications regarding robotics. With a stable operating system is where the project interacts with the ROS2 system, which is utilized to communicate between each part of your system such as control algorithms with the hardware interface.

It essentially acts as a bridge where information can be exchanged between all parts in your system efficiently, enabling your project to scale and remain organized.

Finally, the firmware for the ESP32 was developed using the Arduino IDE. This platform

allowed us to write, compile and upload code efficiently, making it easier to control the servo motor and handle communication with ROS2. Overall, the combination of these tools and hardware for control and movement, and software for design and communication, helped us build a functional and flexible robotic system.

This chapter talked about the ideas of Cyber-Physical Systems and why they are important in things that need computers and physical parts to work together perfectly. It showed how Cyber-Physical Systems have changed over time from controls to complex systems that can make decisions on their own and work automatically using new technologies like the Internet of Things and artificial intelligence.

The project was clearly laid out with an arm system that uses sensors, computers and motors to get precise and fast control. The idea of getting feedback in time and using it to control the system was a big part of this.

The chapter also introduced the parts of the project including the ESP32 microcontroller, servo motors, the mechanical structure and tools like ROS 2 Ubuntu and Arduino IDE. All these things together make up the design of the system.

This chapter gives you the information you need to understand how the system was designed how it was built and how well it works, which will be explained in the chapters, about Cyber-Physical Systems.

CHAPTER 2: LITERATURE REVIEW

This chapter reviews the existing research and technologies related to robotic arms, Cyber-Physical Systems, and intelligent control methods. It explores key concepts such as sensor-less torque estimation, weight-based sorting, digital twin technology, and ROS-based system integration. The chapter also highlights the role of embedded systems and low-cost components in making robotics more accessible. These studies provide the theoretical background and motivation for developing an efficient and cost-effective robotic arm system.

2.1 Sensor less Torque Estimation and Intelligent Control in Robotic Arms

As robotics systems become more intelligent and interactive in modern days, understanding the forces and load exerted on a robot arm becomes a crucial issue. Usually, special force sensors are used for this, which however would increase the costs and complicity of the system.

In order to find solution to this issue, many researchers had been studying the sensor-less approaches to estimate torque and had reported their promising results. [6] Some traditional and well-established methods of sensing torque without using explicit sensors rely on information from other variables related to the applied torque.

For example, current of the motor which bears direct relation to the produced torque, and even tiny deformation in robot joints which might reflect the information of acting force in the system. [6]

A combination of the above methods can achieve reasonable good accuracy of estimating the torques without explicit force sensors, which is very useful to detect the weight-based sorting applications.

2.2 Robotic Arms for Material Handling and Weight-Based Sorting

Material handling is a task that is used by every single industry, mostly warehouses, production lines. Until recently, the task was performed by humans, therefore the task took more time, consisted of repetitive movements and sometimes resulted in errors. Due to industrial demands and need for more efficiency and consistency, automated handling systems have emerged and

robotic arms are frequently used in production processes.

The sorting task performed by the robotic arm has been applied in numerous industrial applications. The sorting procedures that can currently be found in industries have their own classification methods based on colour, shapes and other visual attributes of objects being sorted. [7] There are, however, certain situations where these classification methods are not suitable. An example is an instance when two objects that appear to be identical in colour and shape are actually different from each other when considering their mass. Sorting objects by weight will be more useful in these scenarios.

A weight-based sorting system normally consists of a robotic arm picking an object by a gripper and placing it in a designated location based on the weight of the object. This system can be successfully implemented in applications such as package sorting, logistics and quality control systems in industries. [2] The weight-based sorting system for our project would be suitable due to the reason of distinguishing between the items through a real physical characteristic of the item instead of visual presentation.

The issue of the price also comes into play. A significant proportion of industrial robot systems are too expensive for small-scale companies or student projects. It appears there is demand for systems that could achieve similar task performance at a cheaper price and are simpler to use.

2.3 Digital Twin Technology and ROS-Based Systems

Today, with industries always changing, systems using robots are getting more and more interconnected and smart. In line with this trend, the digital twin concept represents a major milestone, where a replica of the physical system is created in a digital format.

In this way engineers are able to "observe" the behaviour of the machines and investigate them through the simulation models, rather than direct interaction with hardware. The system performance and dynamics can be analysed using simulation instead of real hardware.

By feeding the simulation models with the data obtained in real-time by sensors it is possible to achieve highly accurate representations of the actual system dynamics and the system can therefore be more easily predictable and its anomalies anticipated, giving engineers the possibility to perform the appropriate adjustments beforehand and avoid possible problem at

later stage. With a continuous updating of the simulation model with fresh data the system becomes more "responsive" and upgrades can be achieved seamlessly without constant manual intervention. [8]

The figure 2.1 shows how a digital twin system works by connecting an asset with its virtual representation. Digital twin systems use sensors and connectivity to collect real-time data from systems like machines or processes and send it to the digital twin, which is a virtual model of the physical system. The digital twin system processes this data and analyzes it using advanced analytics and artificial intelligence. The digital twin system also has visualization tools to monitor how things are working. The analysis gives us insights that help us make the digital twin system work better and make decisions. These insights are then used in systems like engineering and maintenance and business and financial platforms, which helps to make things more efficient predict when maintenance is needed and makes the whole digital twin system work better. The digital twin system is very useful, for optimization and decision-making. It helps to improve the performance of the physical system and the digital twin system.

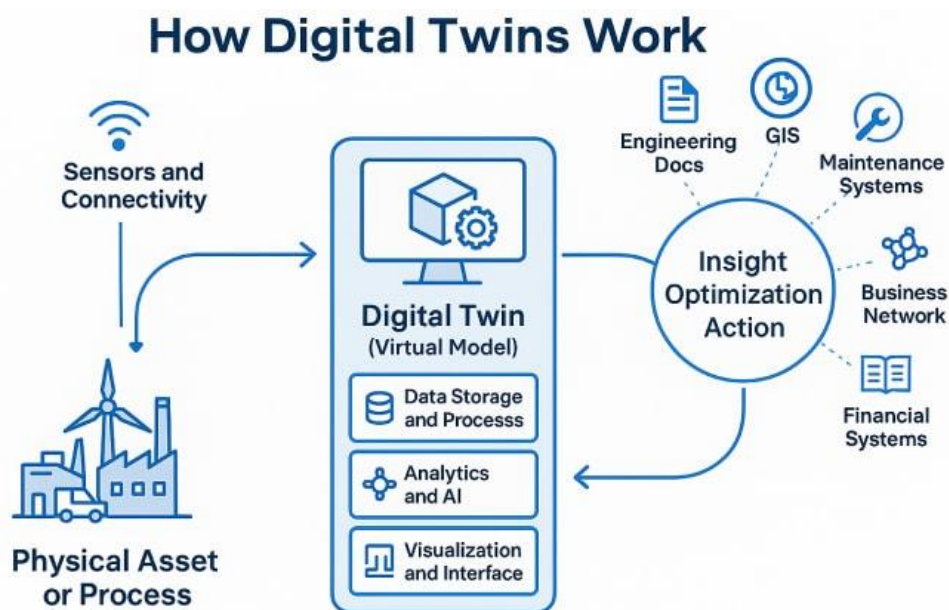


Figure 2.1: Working of Digital Twin

Within such system ROS, as it will be later shown, plays a vital role by managing the communications among the different system components such as sensors, controllers and software modules, making a structured organization within such system, regardless of their growth or possible modifications. [4]

2.4 Cyber-Physical Systems in Smart Manufacturing

An environment-aware system will not have to wait before taking actions. In the moment the software is interacting, decisions are taken, and actions are taken in real time, which a human cannot keep up with. Data will constantly be sent between all the components of the system, similar to how information is transferred in the human body. In the very instant data is received,

the system will react to the changes that have occurred. Simply, the system is like a machine with reflexes, as the actions are dictated directly by the code. Those functions which previously would take a long time to react, can now react instantly.

Research suggests that the overall performance will increase and error reduction will be achieved through automatic correction in cyber-physical systems (CPS). When utilized in robotics, these systems enable machines to perform tasks more precisely and accurately than the methods that were employed previously, which is mainly due to the presence of continuous feedback. [1] This feedback enables the system to learn and modify its actions accordingly as time progresses, thus, by utilizing real-time information, the machine can react instantly to its environment.

The performance of operations becomes greatly improved when dealing with tasks that are variable or have increased complexity. The system reacts instantly to any changes that may occur, as opposed to simply relying on predefined instructions. This enhancement allows the machine to perform tasks more reliably, as it can adapt to any unforeseen scenarios that may arise. The movement is better controlled, since sensors dictate every step of movement. The overall accuracy of the task will be more efficient than prior methods that are governed by steps in the pre-set sequence.

2.5 Embedded Systems and Low-Cost Robotics

The use of embedded systems has been a driving factor in enabling people (students or those with smaller application needs) to readily explore and build robots. Simply, an embedded system is the set of hardware and software used to perform a specific function. In robotics it's used as a controller, normally with microcontrollers such as Arduino or ESP32 being the brains, that controls motors, receives sensor inputs, and manages inter-component communications.

The use of relatively cheap components such as servo motors, breadboards, and jumper wires has made the assembly of working robotic systems affordable, without needing prohibitively expensive hardware. [5]

Servo motors were specifically used due to their ability to control angular movement very accurately and make them ideal for tasks such as grippers or simple manipulators. Components can be readily connected and tested in a short amount of time allowing rapid prototyping and experimenting during the design phase.

Through recent studies it has been shown that such embedded systems can be effectively used for tasks such as pick-and-place actions or object sorting, even using simple hardware. [4]

3D Printing has been another key player in providing an affordable means to build robotics. It's easy to construct bespoke mechanical parts, such as links for a robotic arm, joints, and grippers. [9] This saves costs in manufacturing and provides high levels of flexibility to design custom structures for different projects.

Embedded systems, paired with a range of cheap hardware components, have allowed robotics to become more accessible and scalable. Both economical and efficient systems can be built through this method which is ideal for the educational environment (where the focus is learning and experimenting) as well as smaller commercial applications where price is a primary consideration.

2.6 Objectives of Project

1. Design and development of a robotic arm to perform weight-based sorting tasks.

The first objective of the current study is to develop an automated sorting system utilizing a robotic arm which would perform sorting according to the weight of each object. It will allow the robotic arm to grab the object, determine its approximate weight and categorize accordingly. This objective requires the design of an optimal multi-link robot construction, along with a proper coordination of its joints.

2. To incorporate ROS2 in order to improve coordination and scalability of the system.

Another goal in this robotics project involves incorporation of ROS2 (Robot Operating System 2). This approach will be necessary to control communication processes and system operation.

With its help, a system can be broken down into various nodes or modules for easier development and greater flexibility. Also, by using this technology, robotic arm operation becomes easier to visualize, monitor, and control. Scalability is another goal, which will allow for adding new features or parts to the system at any time.

3. To come up with a cheap way to detect weight differences.

Rather than use special force sensors, which could increase costs considerably, this robotics project aims to devise a cost-effective solution to the problem. In particular, this objective relates to the utilization of motor current in order to determine loads. This means that by measuring motor current consumed when lifting a certain object, it will be possible to estimate torque and then calculate object weight.

4. To design and manufacture the mechanical structure using 3D printing.

As another objective, it is aimed to design and manufacture all mechanical parts of the robot using CAD and manufacture using 3D printing technology. It includes designing the base of the robot, its links and the gripper. Thus, it will be possible to create a mechanical structure which is lightweight, easy to assemble and also allows for customization. Using 3D printing technology will allow us to change the mechanical design in case of any modification needs. At the same time, 3D printing makes manufacturing of all mechanical parts cheap and quick.

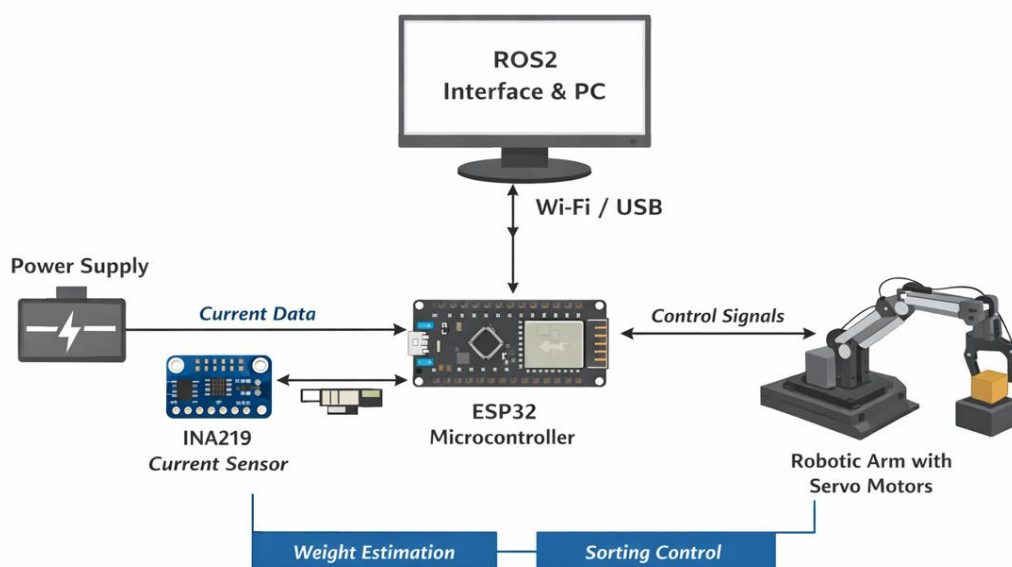


Figure 2.3: Block diagram of weight-based sorting robotic system

The figure 2.3 illustrates the system architecture of the robotic arm Cyber-Physical System. A ROS2 interface running on a PC communicates with the ESP32 microcontroller through Wi-Fi or USB enabling high-level control and monitoring of the robotic arm Cyber-Physical System. The ESP32 microcontroller acts as the central processing unit of the robotic arm Cyber-Physical System receiving data from the INA219 sensor, which measures power consumption from the supply. This data from the INA219 sensor is used for weight estimation of the robotic arm CPS helping determine the load handled by the robotic arm CPS. Based on this information, from the INA219 sensor and the robotic arm CPS control signals are Sent to the servo motors that drive the robotic arm CPS. The robotic arm CPS also performs sorting control linking sensor feedback with actuation for responsive operation of the robotic arm CPS.

5. To showcase the ability to integrate mechanical, electrical and software aspects.

The final project goal is to integrate several aspects of engineering into one complete system. It will require the integration of the mechanical part (the robotic arm), the electrical part (motors and other sensors), as well as the software aspect that utilizes such programs as the Arduino IDE and ROS2. This shows that the area of robotics is highly interdisciplinary in nature.

6. To analyse system performance and suggest areas of improvements.

Finally, the goal of the project is to study the robotic system and test it at various levels to see what its strengths and weaknesses are. As a result of the analysis, it will be easier to define the weaknesses and suggest improvements where necessary.

This chapter talked about the research areas that help with making the proposed system, including ways to figure out torque using robots to sort things and ideas about digital twins. It also said that ROS2 is really important for getting systems to talk to each other and for simulating things and that embedded systems are key to making robots that people can actually afford. The main goals of the project were laid out clearly focusing on designing the system making it cost efficient and getting the hardware and software to work together.

The proposed system is going to be designed and implemented in a way that is cost integrates hardware and software and this is based on the research areas discussed in this chapter, including robotic sorting applications and digital twin concepts and the proposed system will use ROS2 for system communication and simulation. The system and the proposed system are going to be important, in the following chapters, where the design and implementation of the proposed system will be described in detail and the proposed system will be the main focus of the project.

CHAPTER 3: METHODOLOGY

This chapter describes the methodology followed in the design and development of the robotic arm Cyber-Physical System. It explains the step-by-step process used to transform the design into a functional system by integrating mechanical design, hardware implementation and software development. The chapter begins with the design of the arm using CAD tools, followed by the fabrication of components through 3D printing. It then covers the software setup, including the installation of Ubuntu and ROS2 and the creation of the robot model using URDF and XML files. Simulation and validation using RViz are explained to ensure system behavior before hardware deployment. The chapter also presents the approach used for motion analysis through Denavit–Hartenberg transformation.

Finally, it discusses the development of control algorithms using Arduino IDE, integration of sensors such as INA219 and techniques, like torque estimation and Kalman filtering for improving accuracy and system performance of the robotic arm Cyber-Physical System. The robotic arm Cyber-Physical System is developed by following these steps and the robotic arm Cyber-Physical System is the focus of this chapter the development of the robotic arm Cyber-Physical System is explained in detail the robotic arm Cyber-Physical System is a complex system that requires careful design and development.

3.1 Implementation of CPS Layers in the Project

The developed robotic arm system follows the three-layer architecture of a Cyber-Physical System (CPS), namely the Physical Layer, Communication Layer, and Cyber Layer. These layers work together to enable real-time sensing, processing, control, and visualization of the robotic arm.

1. Physical Layer Implementation:

The physical layer of the project consists of all hardware components that directly interact with the real environment. In this project, the physical layer includes:

- 3-DOF robotic arm structure fabricated using 3D printed PLA components.
- SG90 servo motors used for joint actuation.
- ESP32 microcontroller for embedded control.
- INA219 current sensor for monitoring motor current.
- Gripper mechanism for object handling.

The servo motors produce physical movement of the robotic arm links and gripper, while the INA219 sensor continuously measures current consumption during operation. These current values are used for torque estimation and indirect weight detection. The ESP32 receives commands and controls all motors in real time, enabling smooth pick-and-place operations.

2. Network and Communication Layer Implementation:

The communication layer acts as the bridge between the hardware system and the digital environment. In this project, communication is established using:

- ROS2 middleware.
- Serial/Wi-Fi communication between ESP32 and ROS2.
- I2C protocol for INA219 sensor communication.
- ROS2 topics and nodes for data exchange.

The ESP32 sends joint angle and sensor data to the ROS2 environment, where different nodes process and publish information. The Robot State Publisher and Joint State Publisher continuously update transformation frames and joint states. This enables synchronized communication between the physical robotic arm and its virtual model. The communication layer ensures reliable real-time data transfer with minimal delay.

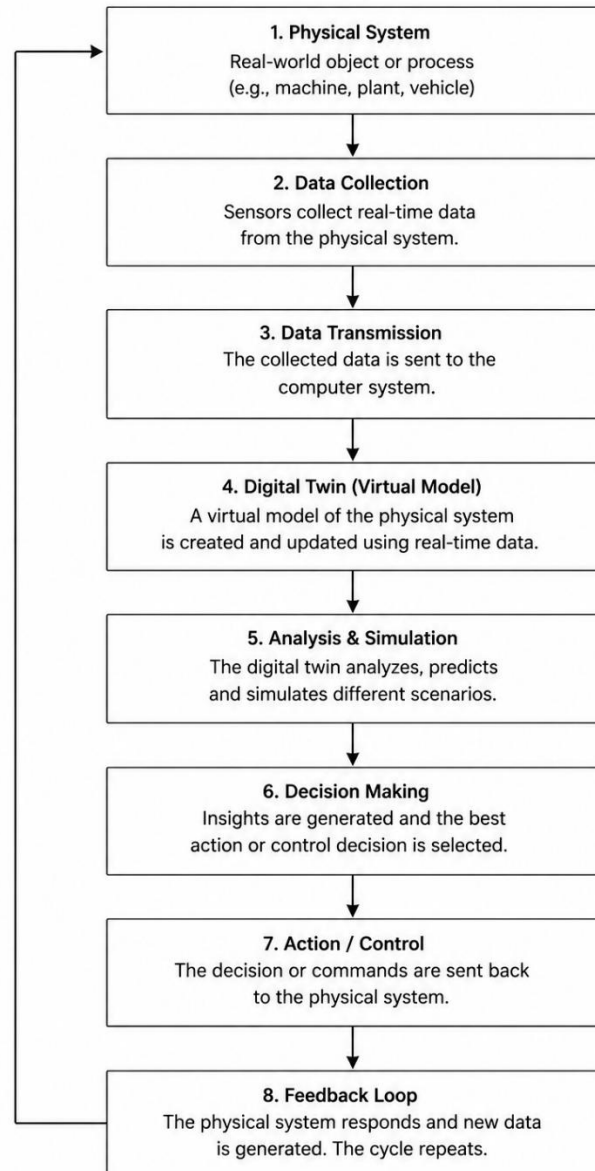
3. Cyber Layer Implementation:

The cyber layer represents the computational and intelligent part of the CPS. In this project, the cyber layer includes:

- ROS2 software framework.
- URDF/Xacro robot modelling.
- RViz digital twin visualization.
- Torque estimation algorithms.
- Kalman filter for sensor data smoothing.
- Control algorithms developed in Arduino IDE and ROS2.

A digital twin of the robotic arm is created using URDF and visualized in RViz. The virtual robotic arm continuously updates according to real-time joint data received from the ESP32. Torque estimation is performed using the motor constant (K_t) and current measurements from the INA219 sensor. A Kalman filter is applied to reduce noise and improve measurement accuracy. The cyber layer processes all incoming data, performs calculations, and generates control decisions for safe and efficient robotic arm operation.

DIGITAL TWIN – SIMPLE FLOWCHART



3.2 Designing of the Robotic Arm

Creating a robotic arm in Fusion 360 was the first stage in completing our project. It was critical since it allowed us to get acquainted with the structure of the device. By modelling, we were able to think through and organize the construction process and not make any mistakes when it comes to fabricating our robotic arm.

Our robotic arm is a 3-degree-of-freedom (3-DOF). This means that it has three axes of movement controlled by three joints. Our structure consists of four main elements: a base, three arm links, and a gripper. All of these elements were modelled separately and combined later into one structure.

The first step in modelling the arm was the creation of its base. In our case, it was necessary for our system to have a proper support, meaning that it should be strong enough to hold all elements of the arm. At this stage, we paid attention to having enough space to install a servo motor on the base.

Once the base was done, the next step involved the design of the arm links. The robotic arm has three arm links, which are connected by joints, allowing the robot arm to have mobility. As we designed the links, we tried to keep them light yet durable enough to carry any object. We ensured the presence of holes to facilitate proper mounting for servo motors as well as screws. Proper length and shape were chosen for the links to ensure sufficient mobility of the arm.

It is important to note that while designing the links, we were particularly concerned with making sure that there was no possibility of collision between two links when operating. This was made easier because Fusion 360 allowed us to assemble all the parts and check their mobility.

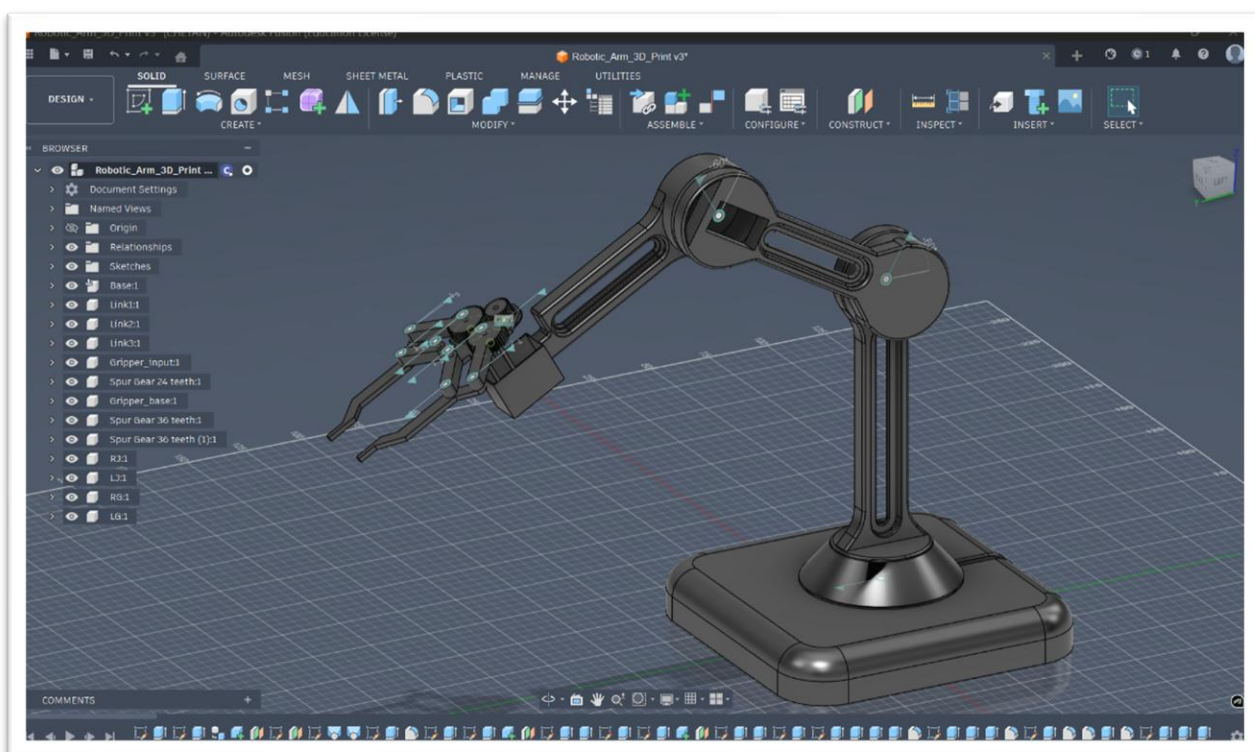


Figure 3.1: CAD Model of robotic arm

Figure 3.1 shows the complete 3D model of the robotic arm made in Fusion 360 with simulations also performed in it.

After that, the design of the gripper was done. Gripper is considered as the most essential component because it allows the robot to grasp and position objects. A simple gripper was designed to function similar to a hand. This gripper consists of two fingers, and it can be opened and closed using a servo motor. During the designing process, we ensured that the gripper had sufficient grip strength to ensure that it doesn't slip from the object. On the other hand, it should not have excessive grip strength that damages the object.

After all components were successfully designed, we integrated them using Fusion 360 software to build the entire robotic arm.

3.3 3D Printing of all Links

After designing the CAD model of the robotic arm using the Fusion 360 application, the following step involved creating the components in reality. In order to accomplish this task, we decided to apply FDM 3D printing, as well as use PLA material. The decision was based on its simplicity of use, low cost, and suitability for manufacturing lightweight and precise parts for the robotic arm.

All components, i.e., the base, arm links, and the gripper were exported to the STL file and sliced using a suitable program. During this process, various parameters related to the printing quality were determined, including layer height, print speed, and infill ratio.

As it is known, the infill ratio influences the strength of the parts produced by a 3D printer. Given that the base serves as the foundation of the whole robot and carries the most loads, its higher density was considered necessary. For this reason, the base was printed with a greater infill ratio than any other part. This helped in ensuring that the base remains rigid and can withstand the forces generated during the movement of the arm.

On the other hand, the arm links were printed with a comparatively lower infill ratio. The main reason for this was to reduce the overall weight of the robotic arm. Lighter links require less torque to move, which improves the efficiency of the servo motors and reduces power consumption. At the same time, the infill ratio was kept sufficiently high to maintain the structural strength of the links and prevent deformation during operation.

The gripper part pieces were printed with a moderate infill rate to make sure that there is a nice balance of rigidity and flexibility. The gripper will need to be rigid enough to grasp objects but light enough to move around smoothly since it is going to touch objects directly.

For printing, PLA plastic was used because it prints relatively easily and does not harm the environment. It is accurate and rigid enough to deal with lightweight objects which is exactly what we need in this project. Furthermore, the problem of warping is minimized, which is why it fits perfectly for mechanical parts such as robotic links and joints.

All the components were printed and later cleaned up from defects such as uneven edges or other minor printing imperfections.

3.4 Ubuntu and ROS2 Setup

For the software aspects of the project, we chose to utilize Ubuntu and ROS2.

ROS2 has known efficiency within Linux systems and running ROS2 alongside Ubuntu would allow for efficient execution within a robot.

To get the software setup we first installed Ubuntu into our system and then upgraded it so that we would be able to access all the libraries and components required and to reduce issues in the future. After installation of Ubuntu we went on to set up ROS2 on our Ubuntu system.

We achieved this by first added the ROS2 repository into the system and installing from the repository. We also added an environment variable so that ROS2 could be accessed in all sessions within our system. We verified the installation of ROS2 by trying out the basic ROS2 publisher/subscriber systems.

After everything had been set up we then set up a ROS2 workspace where all files will be organized for the project. Inside the workspace various packages will be created in order to handle the different functions of the system. ROS2 is useful in this project as it enables communications to take place in the system and because ROS2 has a modular nature we can split our system into modules.

3.5 URDF and XML Files

Once the software setup was done, it was time to make the digital design of the robotic arm that would work according to ROS2 guidelines. For this, URDF or Unified Robot Description Format has been used. It is an XML file that can describe the robot kinematics and geometry.

For describing the model created in Fusion 360 in terms of URDF, each physical part of the robotic arm is described as one link. In our case, we have created five separate links out of it, i.e., the base, three links forming the arm, and finally, the gripper. Along with creating each link, there were visual elements and collision elements for them. Visualization is used to visualize the

robot while performing any tasks, and collisions specify the actual physical boundary for interaction and simulation purposes.

Additionally, we also included the inertial elements to describe the mass and inertia matrix associated with the respective link. Though the current system only requires simple control functions, but having these elements improves the motion behaviour in the simulation. The connections between the links were defined by using joints. Therefore, joints between the links were defined using revolute joints.

The joints therefore establish the relationship between a pair of links and define the motion allowance between the links. The parameterization encompassed the axis of rotation, joint limits concerning the minimum and maximum angles, and the origin, defining the position and orientation of the joint concerning the parent link. They dictate the limits within which the robotic arm moves.

The entire structure was written in XML format, where each component is represented by the use of unambiguously determined tags. For instance, tags were used for rigid bodies while tags described the connections between the bodies. There are tags for geometry types like a box, a cylinder, or a mesh loaded from a CAD file.

As a result, ROS2 can execute parsing and interpretation of the robot model based on this structure. Once the URDF file was completed, it was integrated into the ROS2 workspace and tested using visualization tools such as RViz. This allowed us to verify the correctness of the robot model, including link alignment, joint behaviour, and overall structure.

```

    <collision>
      <origin xyz="-0.071901 -0.047626 -0.132723" rpy="0 0 0"/>
      <geometry>
        <mesh filename="package://Robotic_Arm_3D_Print_description/meshes/Link3_1.stl" scale="0.001 0.001 0.001"/>
      </geometry>
    </collision>
  </link>

  <joint name="Revolute 1" type="continuous">
    <origin xyz="0.0 -0.005 0.02" rpy="0 0 0"/>
    <parent link="base_link"/>
    <child link="Link1_1"/>
    <axis xyz="0.0 0.0 1.0"/>
  </joint>

  <joint name="Revolute 2" type="continuous">
    <origin xyz="-0.001361 0.003761 0.125" rpy="0 0 0"/>
    <parent link="Link1_1"/>
    <child link="Link2_1"/>
    <axis xyz="0.818603 -0.57436 -0.0"/>
  </joint>

  <joint name="Revolute 3" type="continuous">
    <origin xyz="0.073262 0.048865 -0.012277" rpy="0 0 0"/>
    <parent link="Link2_1"/>
    <child link="Link3_1"/>
    <axis xyz="0.818603 -0.57436 0.0"/>
  </joint>
</robot>

```

Figure 3.2: An XML code in Xacro file

Figure 3.2 shows the XML code which used to store data of links , joints and angles of motion.

3.6 Rviz Setup

3.6.1 Simulation in Rviz

Its visualization and verification in the ROS2 environment were the subsequent steps after the creation of the URDF model of the robotic arm. For this purpose, RViz (Robot Visualization Tool), a powerful tool provided in ROS2, was used. It allows viewing the model in 3D without relying on actual hardware, so the structure, movements of joints, and behaviour can be verified using only visualization. The configuration of RViz started by adding the URDF file to the ROS2 workspace. A launch file was created whose purpose was to load the description onto the ROS2 parameter server. This was followed by a robot state publisher node that published transformation (TF) data for each link based on joint states. This transformation data provides the position and orientation of each link with respect to others, thereby providing a complete kinematic chain of the robotic arm. After the robot description had been successfully loaded, RViz was launched in an attempt to visualize the model. In RViz, the display type, 'Robot Model' was added, which offered to render the URDF-defined structure. This resulted in a visualization of the base, links and gripper in 3D space exactly the way they were defined in the URDF file. Other display types like TF (transform frames) were added, which allowed seeing the various frames of coordinates indicating each joint's position and movement in relation to the arm, which is significant in determining how motion is transferred through the arm.

Figure 3.3 and 3.4 show the individual links being imported in the Rviz platform.

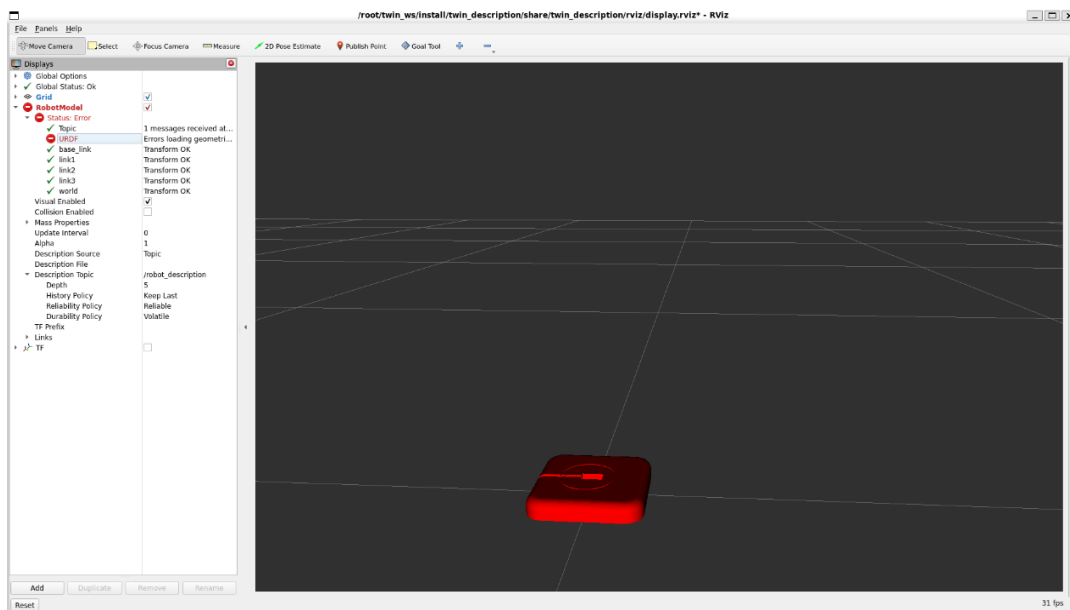


Figure 3.3: Base Link in Rviz

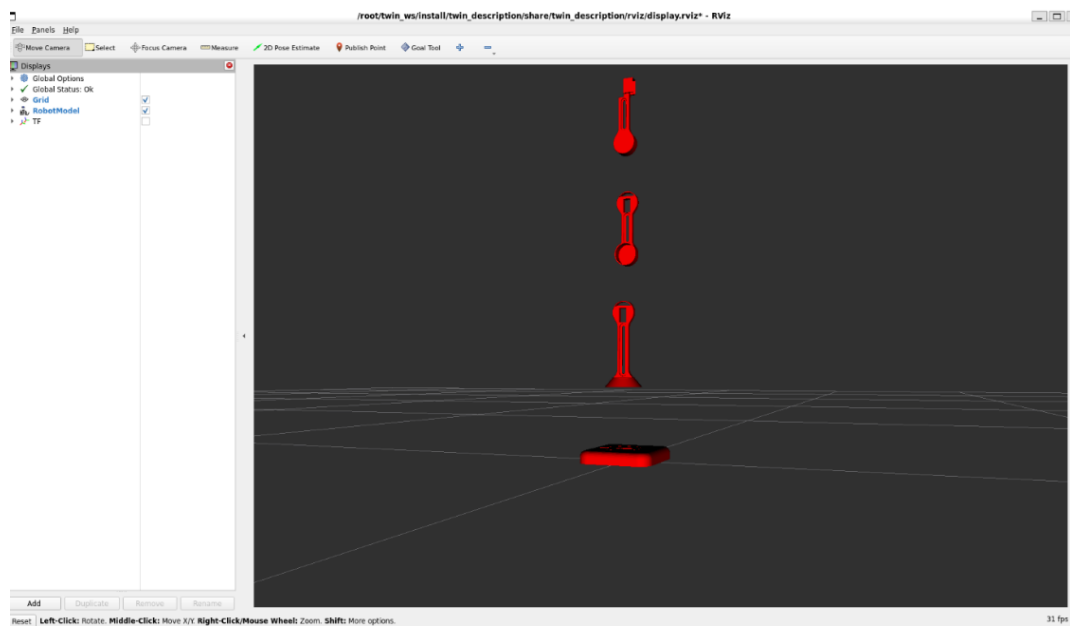


Figure 3.4: Individual Links in Rviz

Figure 3.5 shows the complete assembly of robotic arm with all links imported in Rviz for synchronisation with real robotic arm.

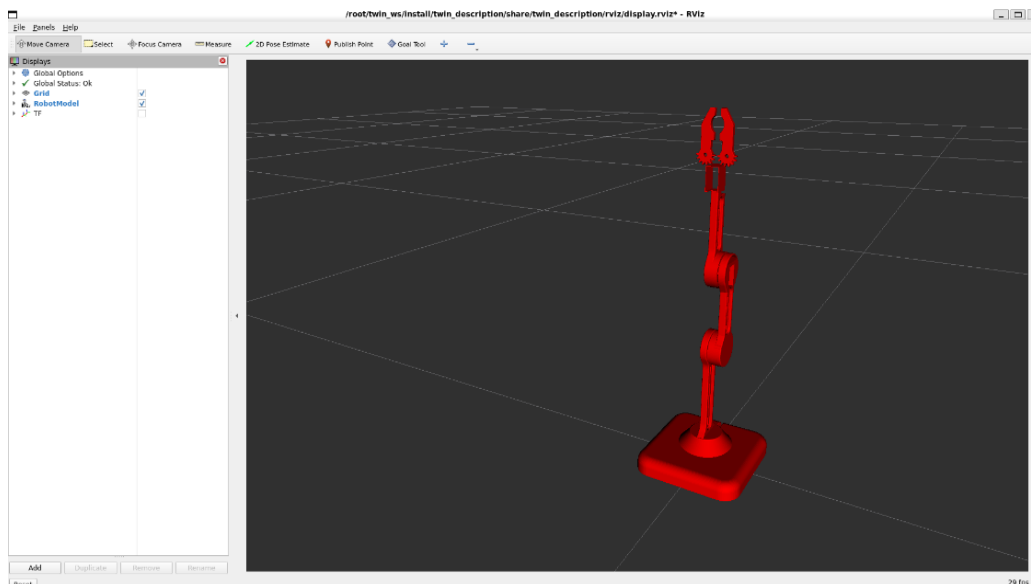


Figure 3.5: Complete robotic arm in Rviz

The publication of joint states would either be performed manually or via a joint state publisher node to simulate movement. Changes in the joint angles helped to see the movement of every link instantly using RViz. This allowed for confirmation that all joints were identified and defined correctly, with correct axis of rotation and limits. Incorrect axis of rotation or unexpected joint movement could be observed and resolved at this point.

3.6.2 Importance of Rviz

The main purpose of using RViz in this project was for the purpose of validation and debugging. Only after that was done was the system connected to the actual hardware. RViz made it possible to verify that the kinematic structure was appropriate and the robotic arm was able to have the desired range of pick-and-place movement.

In addition to validation, RViz is also handy in visualization while the system is running. The data such as joint states, transformations, and other information sent from the robot can be displayed in real-time for easier system monitoring using this tool. This was especially useful in the system integration of ROS2 and the ESP32 controller to test if the commands sent to the robotic arm were working as intended.

Overall, RViz served as an important tool in our development process. It helped bridge the gap between the theoretical design and practical implementation by providing a real-time, interactive visualization of the robotic arm. This not only improved accuracy but also reduced errors during later stages of the project.

3.7 DH Transformation

To control the movement of the robotic arm, there should be a simplified system of locating the motions of each part of the arm. For that purpose, the Denavit–Hartenberg (DH) method was employed. It is a commonly used method to describe the end factor position of each link of the robot step by step.

A robotic arm consists of links which are connected using joints. The joints can be revolute, meaning that it only rotates around the axis. The Denavit-Hartenberg (DH) convention does not take the entire arm into consideration at once but rather focuses on a single joint at a time and then specifies the characteristics of each link in relation to the adjacent link.

For this method, four basic parameters are identified for each joint. These parameters include the length of the link, link twist, the offset of the link, and the joint angle. This is link length is the distance between two joints. The twist of one link with respect to another is given by the link twist. The link offset is the distance along the joint axis, and joint angle defines the rotational movement of the joint. With the help of these four values, the position of each of the parts of the arm of the robotic arm is described.

Figure 3.6 and 3.7 shows the process of doing DH Transformation to find the end factor position of robotic arm.

- θ_i : Joint angle
- d_i : Link offset
- a_i : Link length
- α_i : Link twist

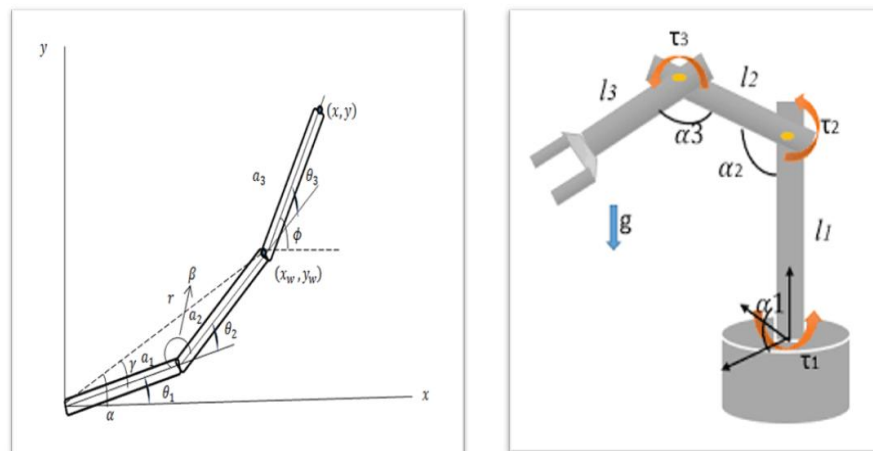


Figure 3.6: DH Transformation

$$T_1 = \begin{bmatrix} \cos \theta_1 & -\sin \theta_1 \cos \alpha_1 & \sin \theta_1 \sin \alpha_1 & a_1 \cos \theta_1 \\ \sin \theta_1 & \cos \theta_1 \cos \alpha_1 & -\cos \theta_1 \sin \alpha_1 & a_1 \sin \theta_1 \\ 0 & \sin \alpha_1 & \cos \alpha_1 & d_1 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

- The overall transformation from base to end-effector:

$$T = T_1.T_2.T_3$$

$$T_2 = \begin{bmatrix} \cos \theta_2 & -\sin \theta_2 \cos \alpha_2 & \sin \theta_2 \sin \alpha_2 & a_2 \cos \theta_2 \\ \sin \theta_2 & \cos \theta_2 \cos \alpha_2 & -\cos \theta_2 \sin \alpha_2 & a_2 \sin \theta_2 \\ 0 & \sin \alpha_2 & \cos \alpha_2 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

$$T_3 = \begin{bmatrix} \cos \theta_3 & -\sin \theta_3 \cos \alpha_3 & \sin \theta_3 \sin \alpha_3 & a_3 \cos \theta_3 \\ \sin \theta_3 & \cos \theta_3 \cos \alpha_3 & -\cos \theta_3 \sin \alpha_3 & a_3 \sin \theta_3 \\ 0 & \sin \alpha_3 & \cos \alpha_3 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Figure 3.7: Homogenous Transformation Matrix

In our project, the robotic arm has 3 degrees of freedom, so we defined these parameters for all three joints. These were taken from the real size and design of the arm links. For this, after defining them, simple mathematical steps (called transformation matrices) were used to find the position of the gripper w.r.t the base.

The main benefit of using DH method is that it makes the movement of the robot easy to understand and calculate. We can follow the same set of steps for every joint instead of using complicated geometry. It is very important to exercise control over the robotic arm to reach the correct position.

3.8 Code Creation in Arduino IDE

3.8.1 Basic code for Servo Motor

Once the design and the working setup of the robotic arm was done, the control over the servo motors by means of Arduino IDE was the next part. It consists of the writing, compiling and uploading the code to ESP32 using the Arduino IDE as it was the controller for the setup.

To begin with a servo motor was controlled using basic program. The basic of servo control was required as that will lead to understanding on how the servo moves in terms of its output based on the input given. It can be noted that servo motor receives PWM signals, and the angle position is changed according to the PWM signal. The desired angle can be achieved by giving the required PWM signal hence in this sense the movement of servo motor is control.

In the program a library which was needed for controlling a servo motor was used. The servo motor was connected to a given GPIO pin of the ESP32. Following this, various values were entered in the program and it caused the motor to move to the position required, normally from 0 to 180. The movement was tested like this and it showed the servo motor was working.

Once the movement was working, it was possible to have a smooth moving effect by adding a delay between the position values. That prevented the motor from moving jerkily. Tests on the servo motor were run for several number of times to ensure the movement accuracy. This step was important as it resulted in the better knowledge of servo motor. Any of the additional motor controls were to be carried out based on this step.

3.8.2 Code for all motors combines with INA219 Sensor

After testing a single servo motor, the robotic arm's all the motors needed to be controlled and connected to the entire robotic system. We connected many servo motors in ESP32, each to control one of the links of robotic arm; base, and the robotic arm link and the gripper. The program was then extended to control many servo motors with the usage of different GPIO pins of ESP32 for each servo. We defined the angles for each of the servo motors to get the desired movement from the robotic arm. Using coordinated movement of all the motors we could achieve simultaneous movement of the links of robotic arm. INA-219 was connected to detect the power usage and current. We used this sensor to detect the power being used and current being delivered from power source to all the motors being used. This was connected via I2C, and the appropriate library was linked with the code.

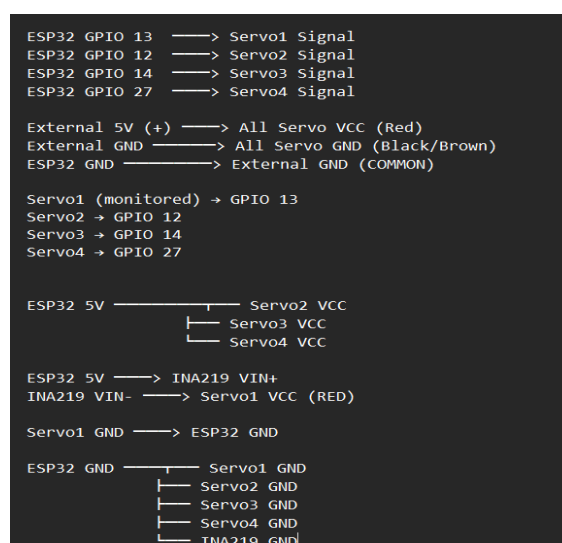


Figure 3.8: Circuit Design representation

Figure 3.8 gives a representation of all the connections between power supply, ESP 32 microcontroller and servo motors of robot.

```
2  #include <Adafruit_INA219.h>
3  #include <ESP32Servo.h>
4
5  Adafruit_INA219 ina219;
6
7  Servo servo1;
8
9  // Pin
10 int servoPin1 = 13;
11
12 // Angle
13 int angle1 = 90;
14
15 // Torque constant (SG90)
16 float Kt = 2.7;
17
18 // Kalman variables
19 float estimatedTorque = 0;
20 float errorEstimate = 1;
21 float errorMeasure = 2;
22 float q = 0.01;
23
24 // Kalman filter
25 float kalman(float measurement) {
26     float kalmanGain = errorEstimate / (errorEstimate + errorMeasure);
27     estimatedTorque = estimatedTorque + kalmanGain * (measurement - estimatedTorque);
28     errorEstimate = (1 - kalmanGain) * errorEstimate + fabs(estimatedTorque - measurement) * q;
29     return estimatedTorque;
30 }
31
32 void setup() {
33     Serial.begin(115200);
34     Wire.begin(21, 22);
```

Figure 3.9: Code for INA219 Integration with circuit

Figure 3.9 gives the code for connections between the current sensor and servo motors with torque constant value and Kalman filter applied in the code.

The readings were then observed for the variation of current as and when the robotic arm tries to lift or move an object. Such kind of detecting variation of current can be used for applications like, weight sorting.

Meanwhile all the 3D printed links were joined with each other to form a robotic arm. All the motors were fixed at each of the joints and they were joined with help of screws and support, with care so that there was no obstacle in moving of any joint.

3.9 Pseudo Codes for Robotic Arm and ROS software

```
BEGIN
// — INITIALIZATION —————
INITIALIZE ROS2 framework
LOAD robot_description ← xacro(twin.urdf.xacro, is_sim=false)
    DEFINE 5 links : base, link1, link2, link3, gripper
    DEFINE 5 joints : joint_1(base), joint_2(lower arm),
                    joint_3(upper arm), joint_4(gripper_L), joint_5(gripper_R)
    SET joint position limits for joint_1 to joint_5
// — START NODES —————
START RobotStatePublisher (robot_description, use_sim_time=False)
START ServoSerialReader (port=/dev/ttyUSB0, baudrate=115200)
START ArmController (bridge: /joint_commands → /joint_states)
START RViz2 (config=display.rviz)
// — MAIN LOOP (20Hz) —————
LOOP forever:
    line ← READ serial from Arduino/ESP32
    PARSE "S1:<v> | S2:<v> | S3:<v> | S4:<v>" → degrees
    CONVERT: rad = (deg - 90) × π / 180
    MAP: joint_1=S2, joint_2=S3, joint_3=S1,
        joint_4=S4, joint_5=-S4 (mimic)
    PUBLISH /joint_states [joint_1 .. joint_5]
    RobotStatePublisher → COMPUTE forward kinematics → PUBLISH /tf
    RViz2 → RENDER digital twin in real time
END LOOP
// — DATA FLOW —————
Physical Arm → Serial(S1–S4°) → Parse & deg→rad
    → /joint_states → Forward Kinematics → /tf → RViz2
END
```

This pseudo code describes how an ROS2-based system updates a real-time digital twin of a robotic arm using data from an ESP32 or Arduino. At the beginning the system is initialized by starting the ROS2 framework and loading the robot description from a Xacro/URDF file this file defines the structure of the robotic arm, including five links, such as the base three arm segments and gripper and five joints. Each joint is assigned motion limits to ensure realistic movement of the robotic arm. The system then launches ROS2 nodes the Robot State Publisher calculates the position of each link based on joint values of the robotic arm the Servo Serial Reader reads angle data sent from the ESP32 via serial communication the Arm Controller acts as a bridge converting incoming commands into joint state messages for the robotic arm and RViz2 is started to visualize the robotic arm in a 3D environment.

The system continuously reads data formatted as servo angles these values are parsed and converted from degrees to radians, which is required for ROS computations of the robotic arm the values are then mapped to corresponding joints, including a mirrored movement, for the gripper fingers of the robotic arm. The processed joint values are published to the /joint_states topic the Robot State Publisher uses these values to compute kinematics and updates the transformation frames, RViz then uses this data to render the digital twin of the robotic arm in real time. The data flow shows how physical movement of the arm is captured, processed and visualized the robotic arm sends angle data the data is converted and published kinematics are computed and the digital model of the robotic arm is updated accordingly.

Pseudocode: Robotic Arm Control System

BEGIN

INITIALIZE I2C, INA219 sensor, and 4 servo motors

SET initial angles (a1, a2, a3, a4) and parameters (Kt, armLength, delay, Kalman variables)

MOVE servos to home position

LOOP forever:

FOR each predefined position:

MOVE servos sequentially to target angles

AT each step:

READ current

COMPUTE $\text{torque} = K_t \times \text{current}$

APPLY Kalman filter to torque

COMPUTE force and weight

PRINT all values

PRINT "Base moving according to weight"

IF weight < threshold THEN

MOVE base to left position

ELSE

MOVE base to right position

ENDIF

```
    MOVE arm through return positions
    RETURN to home position
END LOOP
```

```
END
```

The pseudocode outlines a robotic arm system that begins by initializing I2C communication, an INA219 current sensor module, and four servo motors. It sets initial angles and parameters such as the torque constant, arm length, delay, and Kalman filter variables, then moves the arm to a home position. The system then enters an infinite loop where it cycles through predefined positions, moving the servos step by step. At each step, it measures current, calculates torque, applies a Kalman filter to smooth the data, and computes force and weight, printing these values for monitoring. Based on the calculated weight, the system decides the direction of base movement shifting left if the weight is below a set threshold or right if it exceeds it. After completing the movement sequence, the arm passes through return positions and returns to the home position before repeating the entire process continuously.

3.10 Use of INA219 Sensor

We have used INA219 sensor in this project to measure the current flow through the motors. This sensor is useful because it can measure the voltage and current in real-time and send the data to the controller via the I2C communicator.

The INA219 sensor is connected in series with the power supply of the motors, thereby enabling it to have real-time data on the current being drawn. If the robotic arm does not carry any load, the motor will draw a smaller current. However, when the arm lifts or moves an object, especially a heavier one, the motor needs more power which in turn increases current. Now this change in current becomes an essential input to our system. Instead of directly measuring weight (which would require a separate sensor), variation in current value is used as indirect means to understand how much load the motor is taking.

This removes the necessity for a separate weight sensor, ensuring the system remains compact, cheap and reliable without losing the essence of using weight as a discriminator.

3.11 Torque Estimation using Motor Constant (K_t Principle)

Once we have the current values from the sensor, the next step is to relate this current to the torque produced by the motor. This is done using the motor constant, known as K_t . The value of K_t is specific to a motor and tells us how much torque is generated per unit of current.

The basic idea is simple: **more current means more torque.**

When the robotic arm lifts a heavier object, the motor needs to produce more torque, and therefore it draws more current. By multiplying the measured current with the motor constant (K_t), we can estimate the torque produced by the motor.

This method helps us understand how much effort the motor is putting in at any moment. In our project, this information is useful for identifying differences in object weight. For example, if the current suddenly increases when picking an object, it indicates that the object is heavier compared to others.

3.12 Use of Kalman Filter for Data Smoothing

As already mentioned, there are practical issues with sensor readings. Readings currently displayed on the INA219 will have very small fluctuations or 'noise' due to possible variations in electric supply, the choppy movement or variations in motor action.

To get over this problem, a Kalman filter has been applied. Simply put, a Kalman filter is a group of equations that increases accuracy of readings obtained from sensors by filtering out any electrical noise. This is done by using information already processed rather than readings taken directly, hence results in smoother values that can be analyzed more accurately.

To put simply, a Kalman filter combines current readings with any prior readings in order to create a 'better' reading, i.e. Electrical noise has been discarded. In our case, the Kalman filter applied to our current readings will result in smoother estimation of torque values, which will be beneficial in our weight-based sorting, as any large fluctuations will result in wrongly estimated value and may possibly lead to misclassification.

This chapter detailed the methodology used to develop the robotic arm system covering both hardware and software aspects. It described the design and fabrication of the structure using Fusion 360 and 3D printing ensuring a balance between strength and weight. The setup of Ubuntu and ROS2 enabled communication, development and system organization while URDF modeling and RViz simulation helped verify the structure and movement of the robotic arm before physical implementation.

The chapter also explained the use of Denavit–Hartenberg transformation for analyzing arm movement and determining the position of the end effector of the robotic arm. Control of the arm was achieved through Arduino-based programming of the ESP32 with multiple servo motors working in coordination.

The integration of the INA219 sensor allowed time current measurement, which was used for indirect weight estimation through torque calculation of the robotic arm. The application of a Kalman filter improved data accuracy by reducing noise in sensor readings of the arm system.

CHAPTER 4: RESULTS AND DISCUSSIONS

This chapter presents the results obtained from implementing the robotic arm Cyber-Physical System. The systems performance under operating conditions is discussed. The focus is, on validating the design, control and integration of both hardware and software components of the robotic arm Cyber-Physical System.

The ability of the robotic arm Cyber-Physical System to perform movements is examined. The robotic arm Cyber-Physical Systems ability to execute pick-and-place operations is also examined in detail.

The chapter evaluates the effectiveness of sensor integration. Data processing techniques are also evaluated. This includes torque estimation using the motor constant. Time current measurement is used for torque estimation.

Observations related to the arm Cyber-Physical Systems behaviour are discussed. The stability of the robotic arm Cyber-Physical System and accuracy are discussed. This helps understand how well the robotic arm Cyber-Physical System performs in scenarios.

4.1 Mimic the pick and place action

One of the first things we achieved in this project was getting the robotic arm to move properly with all its links. The arm has three degrees of freedom and could perform coordinated movements as planned. Each joint responded well to the control signals, allowing the links to rotate and position themselves in the needed direction.

Overall, the arm's motion felt controlled and stable, showing that the design and programming worked well together. When we tested the system, the arm could execute a basic pick-and-place operation. It could move toward an object position, the gripper closed and opened after the base link motor rotated at a certain angle.

Figure 4.1 shows the final assembly of robotic arm with all connections with sensors and servo motors.

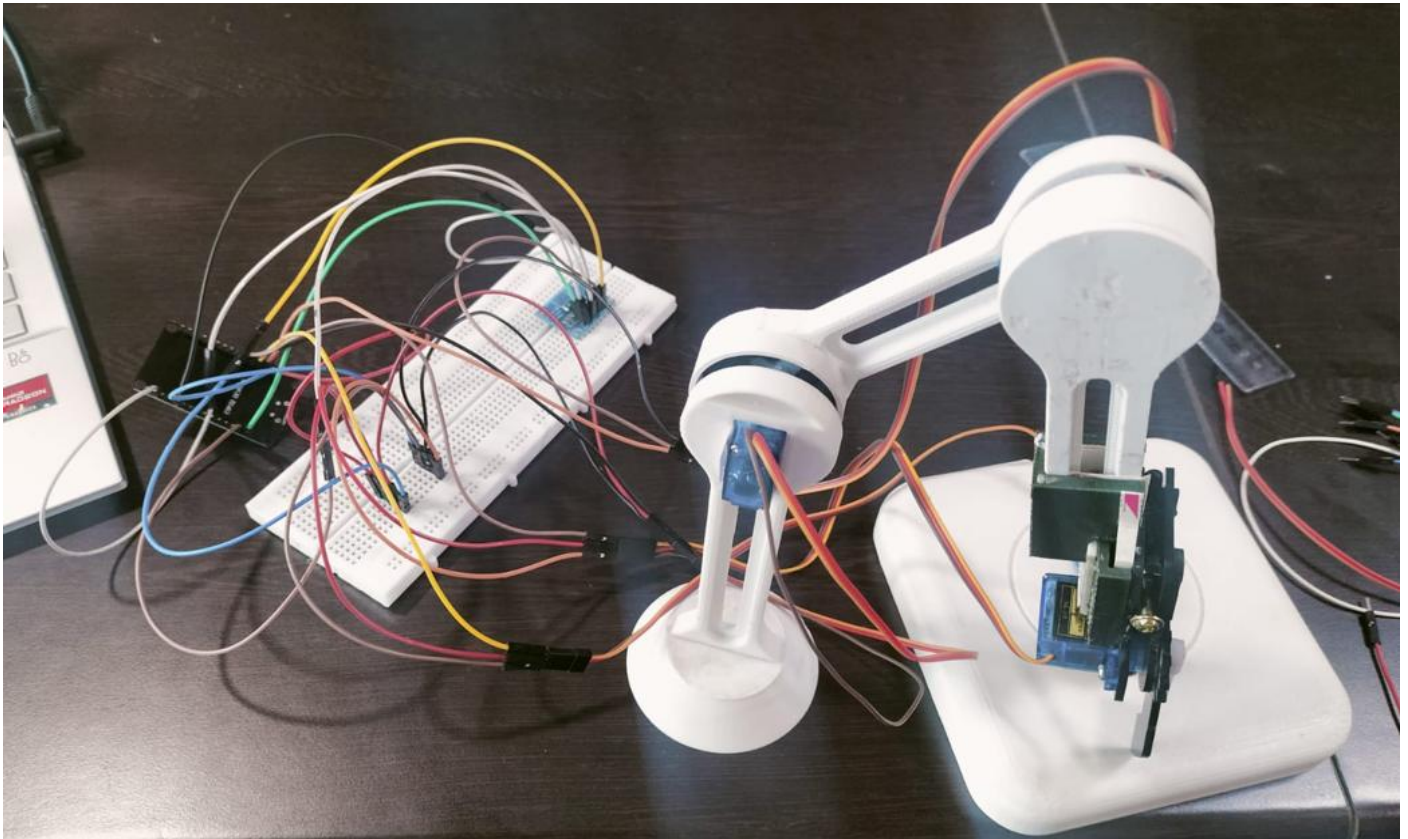


Figure 4.1: Assembly of Robotic Arm

All these movements had to be controlled with every single one of the joints and the result was very motivating as it was clear that the system was able to achieve the movements smoothly.

The gripper was able to stably grasp small objects and the joints showed enough mobility to move the arm to various position without getting stuck. One minor observation while the arm was moving was that some oscillations can be noticed at some angles. This is perfectly normal since the arm is built from light weight 3D printed materials and it moves from small servo motors, but as it was stated, did not create a major problem to the robot.

The robot can mimic the basic pick and place movements which is fundamental for an automated system and the demonstration indicates that the design, assembly and control techniques are correctly implemented. As a result of this success, more features will be added such as the weight detection and separation.

4.2 Measuring Torque values with Kt constant

Another integral part of this project was estimating the torques generated by our motors. We were able to achieve this with the help of the motor torque constant (K_t).

Instead of relying on an additional torque sensor, we utilized the current values from the INA219 sensor and the principle that motor torque is directly proportional to the amount of current that the motor draws, and calculated the torque generated at different points of arm movement. The current from the INA219 was multiplied with the K_t value of the motor to estimate the torque.

During testing we observed that, if load on the robotic arm was increased, the current drawn from the motor increases. This could be seen directly in the values of torques we had computed.

As we lifted lighter objects the current drawn from the motor was minimal, so our computed values of torque were very low too. And when we lifted heavier objects the increase in current led to increase in the calculated values of torque.

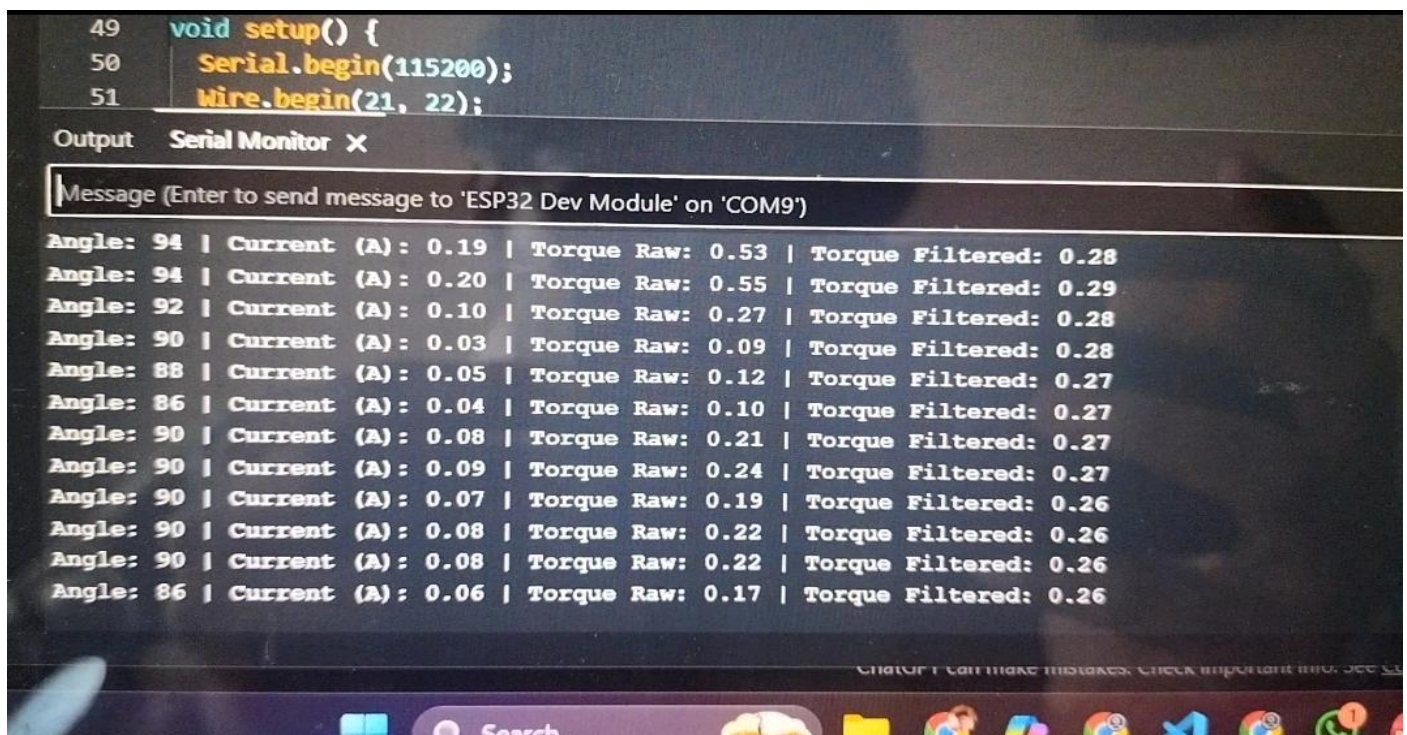


Figure 4.2: Torque values from K_t

Figure 4.3 shows the live or real-time readings of the current and torque while performing pick and place action.

These values might not be exact but they are close enough to allow us to differentiate between categories of weight. This was enough for our application since we were more interested in categorizing according to a difference in weight rather than knowing the exact torque. This method was extremely cost effective and simple.

Figure 4.4 shows the readings of current , torque and estimated weight of the object being lifted.

	Movement in	current(A)	Torque(with kt 2.7)	weight(gm) with arm length 17 cm
Normal	Base	0.02	0.054	32.37992445
	Gripper	0.03	0.081	48.56988667
	motor 1	0.06	0.162	97.13977334
weighted(15g)	Base	0.05	0.135	80.94981112
	Gripper	0.07	0.189	113.3297356
	motor 1	0.13	0.351	210.4695089
float torque = Kt * current_A;				
float force = filteredTorque / armLength;				
float weight_kg = force / 9.81;				
float weight_g = weight_kg * 1000;				

Figure 4.3: Readings from the code

This chapter demonstrated that the robotic arm system successfully achieved coordinated motion and was able to perform basic pick-and-place operations with stable and responsive control. The integration of multiple servo motors allowed smooth movement across all joints, while the gripper effectively handled objects with sufficient precision. Minor oscillations were observed due to the lightweight structure, but they did not significantly affect system performance.

The use of the INA219 sensor combined with the motor constant enabled torque estimation without the need for additional sensors. Variations in current provided a reliable indication of load changes, allowing the system to differentiate between objects of different weights. The real-time data processing and visualization through ROS2 further confirmed proper system operation.

CHAPTER 5: CONCLUSION AND FUTURE SCOPE

This chapter presents the conclusion of the project and discusses possible future developments. It reviews the overall performance of the robotic arm system and reflects on how the design, implementation, and testing met the intended objectives.

5.1 Conclusion

The project was successful in designing and constructing a low-cost robot arm which could be used for simple automation. The 3-DOF robotic arm performed joint movements well and confirmed that the design of the mechanism, 3-D printed components, and control system worked together correctly. Controlled movements were also executed with fluidity, allowing it to be used in simple manufacturing and learning applications.

The robot arm also successfully replicated the pick and place motion, which showed that the joints worked well. The links are light and that a stable base allowed for smooth motion, although minor vibrations were still visible.

A key achievement of this project was the implementation of weight-based sorting using an indirect method. Instead of using a dedicated weight sensor, the system relied on current measurements and torque estimation. The following points summarise this part:

- Current drawn by the motor increased with increase in load.
- Torque was estimated using the K_t (motor constant) principle.
- The method proved to be simple and cost-effective.

Integrating software tools was also a significant part of the project. The use of ROS2, and with it URDF and RViz greatly aided in visualizing the robotic system. This aided in organizing the system, making it simpler to test and debug. The overall systems were a great illustration of how modern robotics integrates software and hardware elements.

However, some limitations were observed during testing. These include:

- Small fluctuations in sensor readings due to noise.
- Limited accuracy in torque estimation for very small weight differences.
- Minor vibrations due to lightweight structure.

- Limited load-carrying capacity of the robotic arm.

Overall, the project achieved its main objective of developing a functional robotic arm capable of performing weight-based sorting. It shows that even with low-cost components, it is possible to build an efficient and intelligent system.

5.2 Future Scope

While the current project is a successful proof-of-concept showing a basic robotic arm that can sort by weight, there are endless ways for improvement. With a couple enhancements, the system can easily become more intelligent, accurate, and ready for real-world applications:

One of the biggest advancements the system could achieve would be the incorporation of an AI-based camera system. If a smart camera and the proper image processing are included, the robotic arm will be able to recognize objects visually as well as determine their weight:

- Smart cameras and object recognition can be used on systems such as OpenCV, and basic machine learning models to allow recognition of object by shape, size, color etc.
- The robotic arm will be able to sort by more than just weight. The robot will now have multi-parameter sorting ability using both weight and visual input.
- The overall accuracy of decisions will increase substantially.

There are ways to make the sensing aspect of the project more efficient as well. The current system of using motor current to predict weight is inherently inaccurate.

- The addition of load cells or other force sensors could easily provide highly accurate measurements for weight estimation.
- Using AI models the system could be trained to predict weights more accurately using data from these sensors.

It is also worth mentioning some of the mechanical aspects that can be improved.

- The degrees of freedom (DOF) on the robotic arm can be increased to allow for more complex movements.
- Stronger materials can be used so the robot can handle a larger quantity of weight.
- A more advanced gripper could be created that could accommodate different shapes and sizes of objects.

The control of the robot can also be enhanced considerably:

- Multiple arms can be controlled together using the control system.

This chapter was about a project that made an arm. The robotic arm was special because it was cheap to make and it could move its joints smoothly. It could also pick up things, put them down in different places. The robotic arm could even sort things based on how they weighed.

A combination of mechanical design, embedded control and software tools were used to make it work. They found out that this combination was very effective. However, we also found some problems with the arm. Sometimes it would vibrate a bit and it would not always sense things correctly.

The robotic arm project showed that there are some things that can be improved. For example, the robotic arm could be made accurate if it had better ways of sensing things. It could also be made reliable if it had smarter control techniques. The project, with the arm was a success and it can be made even better with some improvements.

REFERENCES

1. Morella, P., Lambán, M.P., Royo, J.A., Sánchez, J.C., Muñoz, O.: Cyber Physical Systems implementation to develop a Smart Manufacturing. IOP Conf. Ser.: Mater. Sci. Eng. 1193, 012114 (2021). <https://doi.org/10.1088/1757-899X/1193/1/012114>.
2. Du, H., Yang, G., Pei, L.: Design and Algorithm Research of a Weight Sorting Transport Robot. Manufacturing Technology. 23, 967–975 (2023). <https://doi.org/10.21062/mft.2023.113>.
3. Yan, C., Lyu, M., Chen, Y., Zhang, J.: An Adaptive External Torque Estimation Algorithm for Collision Detection in Robotic Arms. Sensors. 25, 6315 (2025). <https://doi.org/10.3390/s25206315>.
4. Hernandez-Mendez, S., Maldonado-Mendez, C., Marin-Hernandez, A., Rios-Figueroa, H.V., Vazquez-Leal, H., Palacios-Hernandez, E.R.: Design and implementation of a robotic arm using ROS and MoveIt! In: 2017 IEEE International Autumn Meeting on Power, Electronics and Computing (ROPEC). pp. 1–6. IEEE, Ixtapa, Mexico (2017). <https://doi.org/10.1109/ROPEC.2017.8261666>.
5. Ramish, Hussain, S.B., Kanwal, F.: Design of a 3 DoF robotic arm. In: 2016 Sixth International Conference on Innovative Computing Technology (INTECH). pp. 145–149. IEEE, Dublin, Ireland (2016). <https://doi.org/10.1109/INTECH.2016.7845007>.
6. Gold, T., Völz, A., Graichen, K.: External Torque Estimation for an Industrial Robot Arm using Joint Torsion and Motor Current Measurements. IFAC-PapersOnLine. 52, 352–357 (2019). <https://doi.org/10.1016/j.ifacol.2019.11.700>.
7. Ali, Z., Sheikh, M.F., Al Rashid, A., Arif, Z.U., Khalid, M.Y., Umer, R., Koç, M.: Design and development of a low-cost 5-DOF robotic arm for lightweight material handling and sorting applications: A case study for small manufacturing industries of Pakistan. Results in Engineering. 19, 101315 (2023). <https://doi.org/10.1016/j.rineng.2023.101315>.
8. Chinnasamy, S.K., Sura, H.P., Saleem, A., Kathirvel, A., Rangan, P.: Digital twin of robot manipulator using ROS. Presented at the IV INTERNATIONAL SCIENTIFIC FORUM ON COMPUTER AND ENERGY SCIENCES (WFCES II 2022) (2023). <https://doi.org/10.1063/5.0178239>.
9. Almobaied, M., Abu Metair, A., M. Thabet, B.: Prototype of 3-DOF Pick and Place Arm Robot. IRJIET. 07, 01–09 (2023). <https://doi.org/10.47001/IRJIET/2023.703001>.